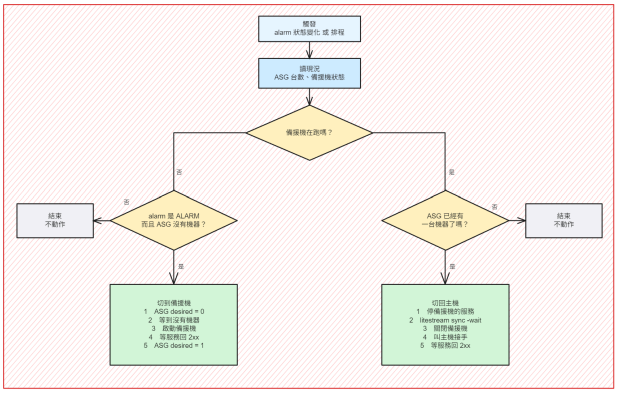
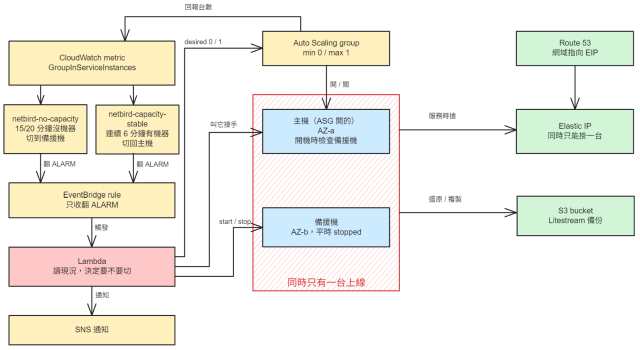




Alarm

netbird-no-capacity

```
metric              GroupInServiceInstances
comparison          LessThanThreshold
threshold           1
period              60
evaluation_periods  20
datapoints_to_alarm 15
treat_missing_data  breaching
```

最近 20 分鐘裡有 15 分鐘沒機器就翻 ALARM，觸發切到備援機，另外發 SNS。

設 15/20 不是 15/15 是因為嚴格要求連續 15 筆的話，spot 反覆開起來又被收回的情況永遠湊不滿，每次成功開機都會把連續計數打斷，但那就應該要開始算次數了

netbird-capacity-stable

```
metric              GroupInServiceInstances
comparison          GreaterThanOrEqualToThreshold
threshold           1
period              60
evaluation_periods  6
datapoints_to_alarm 6
treat_missing_data  notBreaching
```

連續 6 分鐘都有機器就翻 ALARM，觸發切回主機，不發 SNS。

這個 alarm 的語意是反的，它停在 ALARM 代表容量穩定。6 分鐘就是防抖動的門檻，spot 開起來撐不到 6 分鐘就不會觸發切回主機，備援機也就不會被停掉

Litestream

備份指令

用下面的指令備份

```
litestream sync -wait -timeout 30 /path/to/db
```

加 `-wait` 會阻塞到遠端複製完成才回傳，不加的話只是叫 daemon 開始同步就回來了。三個 db 各跑一次，看回傳碼判斷成不成功。切回主機靠這個確認備援機的資料真的進 S3 了，才會把備援機關掉

spot 被回收要不要備份

litestream 持續複製所以最多掉一個 sync 間隔，但中斷有 2 分鐘警告，所以其實還是可以備份
後來用 ExecStop 讓他在停止服務前先做一次備份

S3 要不要版控

看看開 versioning 會不會很多。現在 7 天 125 個物件共 1 MB，最大單檔是 store.db 快照 30 KB，寫入集中在開機，也不是 log 那種一筆一個物件。感覺應該不會到很大，但還是要配 lifecycle 把舊版本過期掉

重複寫入的問題跟可能可以的解決方法

他的同步方式是，每次同步會比對本地的 db_txid 跟遠端的 replica_txid，落後的話先把遠端的 L0 抓下來補自己的狀態，然後把自己現在的資料庫狀態當成下一個 txid 推上去

實驗結果是最後同步的那台會整個蓋掉另一台 (就很像 git push --force)，所以不能兩台同時寫

lambda

被叫起來之後直接重新讀一次現況，查 ASG 有幾台、備援機是 running 還是 stopped，再決定要做什麼

切換流程

今天把流程寫進 user_data，最大的改動是把「開機」跟「開始服務」拆開。

以前開機就等於開始服務，`associate-address`、`docker compose up -d`、`systemctl enable --now litestream` 都是無條件執行。問題出在缺貨結束的時候。前面說的故障是 AWS 沒有 spot 機器可以給，ASG 一直開不起來所以切到備援機。等 AWS 那邊又有閒置機器了，ASG 就會成功開出一台，那台開機的瞬間就搶走服務，可是備援機還在跑，兩台會同時寫同一個 S3。

改成三個腳本。`netbird-prepare` 只做準備，下載 config、`compose create`、還原三個 db。`netbird-takeover` 才真的開始服務，搶 EIP、`compose up -d`、啟動 litestream。`netbird-standdown` 反過來，先停 NetBird 再 flush 再停 litestream，順序反了會漏掉最後那段寫入。

開機時查備援機的電源狀態決定要不要接手，用 tag 查而不是寫死 instance id。

另外發現切回主機的順序本身有問題，本來先停備援機再叫主機接手再驗證，萬一主機起不來備援機已經沒了只能人工救。改成先接手、驗證通過、才停備援機，中間備援機的 NetBird 已經停了不會兩台同時寫，但機器還開著所以退得回去。

情況一：spot 被換掉，ASG 直接補一台

最常見的情況，全自動，Lambda 完全不參與。

```
觸發    AWS 回收 spot，或 EC2 健康檢查失敗

1  關機時 netbird-flush.service 的 ExecStop 跑 netbird-flush
   三個 db 各一次 litestream sync -wait，確保最後的寫入進 S3
```

```
2  ASG 終止舊機器
3  ASG 開一台新的，cloud-init 跑 user_data
4  裝 docker / compose / litestream
5  netbird-prepare    下載 config、compose create、還原三個 db
6  查備援機，是 stopped 就執行 netbird-takeover
7  搶 EIP、等 DNS、compose up -d、啟動 litestream、等憑證、備份 config
```

實測整個替換 75 秒，兩個 alarm 都不會造成動作，metric 只掉 1 到 2 筆不會觸發 no-capacity 因為要 15 筆。capacity-stable 會翻 OK 再過 6 分鐘翻回 ALARM，那次會叫醒 Lambda (cloudwatch 會叫他)

情況二：spot 很久開不起來，切到備援機

```
觸發    AWS 沒有 spot 容量，ASG 一直重試失敗
        backoff 實測 65、122、243、485、954 秒，而且不會發任何事件

1  metric 持續 0
2  最近 20 筆有 15 筆是 0，no-capacity 翻 ALARM（15 分鐘門檻加上約 2 分鐘評估延遲）
3  EventBridge rule 比對 alarmName = no-capacity 且 state.value = ALARM，
   比對成功就直接呼叫 Lambda。rule 不只是過濾器，它同時是呼叫者
4  Lambda 重讀 ASG 台數跟備援機電源，走切到備援機
     步驟 1  ASG desired = 0                  關掉配對
     步驟 2  等 ASG 的 Instances 陣列清空     取得互斥
     步驟 3  start-instances 備援機
     步驟 4  等 HTTPS 回 2xx                  確認服務真的在備援機上
     步驟 5  ASG desired = 1                  ASG 繼續找
5  SNS 通知
```

步驟 2 等的是 `Instances` 清空不是 metric 歸零，因為 Pending 的機器不算 InService，metric 已經是 0 但它的 user_data 正在跑，隨時會搶 EIP。

備援機接手的方式跟主機不一樣，主機是 ASG 每次開一台新機器，cloud-init 會跑 user_data，判斷邏輯放在裡面就好。備援機是同一台機器反覆開關，user_data 只有第一次建立時會跑，之後 `start-instances` 不會再跑，所以要靠一個 systemd unit `netbird-takeover.service`，它 enable 之後每次開機都會執行 `/usr/local/bin/netbird-takeover`。主機不 enable 這個 unit，因為主機要先查備援機狀態才能決定接不接手。備援機可以無條件接手是因為 Lambda 只在確認 ASG 沒機器之後才去開它，被開機本身就是接手的指令。

情況三：容量回來，切回主機

```
觸發    ASG 終於開出一台 spot

1  新機器跑 user_data，netbird-prepare 把東西準備好
2  查備援機，是 running 所以停在這裡，不搶 EIP 也不啟動 NetBird
   它是 InService 但不服務，是個準備好的接手者
3  metric 變 1，連續 6 筆之後 capacity-stable 翻 ALARM（6 分鐘加上約 2 分鐘延遲）
4  EventBridge rule 比對 alarmName = capacity-stable 且 state.value = ALARM，
   比對成功就直接呼叫 Lambda
5  Lambda 走切回主機
     步驟 1  備援機沒在跑就結束                平常最常走的路徑(只是普通的切換 ec2)
     步驟 2  netbird-standdown 停備援機         compose stop、flush、停 litestream
     步驟 3  litestream sync -wait 確認送完
     步驟 4  send-command 叫待命主機跑 netbird-takeover
     步驟 5  等 HTTPS 回 2xx                    失敗到這裡都還能回滾
```

```
步驟 6   stop-instances 備援機          不可逆點在這裡
步驟 7   通知
```

回滾是先叫主機(ASG 這邊)停 NetBird、等確認停了、再把 EIP 搶回備援機、重啟它的 NetBird。第一步不能省，因為步驟 5 逾時不代表主機沒起來可能只是慢，不先叫它停就把備援機拉回來會變成兩台同時寫。

服務空窗是步驟 2 到步驟 5。主機已經開好待命只要跑接手腳本，比全新開機的 1 分 51 秒短。

還沒做

- 備援機那台 EC2 還沒建

小結

今天的東西都繞著同一個限制，litestream 不能兩台同時寫。實驗出來的行為是最後同步的那台會把另一台整個蓋掉，所以主機跟備援機在任何時候都只能有一台在服務。

把開機跟開始服務拆開就是為了這件事。以前 user_data 一跑就搶 EIP、compose up、啟動 litestream，缺貨結束 ASG 開出新機器的那一瞬間就會變成兩台一起寫。拆成 prepare 跟 takeover 之後，新機器可以先把 config 下載好、db 還原好，停在準備好但不服務的狀態，等 Lambda 確認過才接手。standdown 的順序也是同一個道理，先停 NetBird 再 flush 再停 litestream，反過來會漏掉最後那段寫入。切回主機的順序也改了，本來是先停備援機再叫主機接手，主機起不來就只能人工救，改成先接手、驗證 HTTPS 回 2xx、才停備援機，不可逆的那一步放到最後。

兩個 alarm 的參數是被 spot 的實際行為決定的。no-capacity 用 15/20 而不是 15/15，因為 spot 反覆開起來又被收回的時候，每次成功開機都會把連續計數打斷，嚴格連續永遠湊不滿。capacity-stable 的語意是反的，停在 ALARM 代表容量穩定，6 分鐘的門檻是拿來防抖動，spot 撐不到 6 分鐘就不會把備援機關掉。

Lambda 被叫起來之後不看事件內容，直接重新查 ASG 幾台、備援機是 running 還是 stopped 再決定做什麼，所以同一個 alarm 重複觸發也不會做錯事。備份那邊確認 litestream sync 要加 -wait 才會等遠端複製完成，不加只是叫 daemon 開始同步就回來，切回主機前就是靠它確認備援機的資料真的進 S3。spot 被回收有 2 分鐘警告來得及備份，所以改成用 ExecStop 在停服務前先做一次。S3 開 versioning 看起來不會太大，7 天 125 個物件總共 1 MB，但還是要配 lifecycle 把舊版本過期掉，最後決定先不做。

還沒做的是備援機那台 EC2 還沒建，所以情況二跟情況三還沒辦法真的跑一次，目前實測過的只有 ASG 自己補機器的測試，整個替換 75 秒。

參考資料

- Command: sync - Litestream(`-wait` / `-timeout`)
- How it works - Litestream
- Running as a Systemd service - Litestream
- systemd.service - ExecStop
- Spot Instance interruption notices - Amazon EC2(2 分鐘警告)
- Alarm evaluation - Amazon CloudWatch(M out of N)
- Configuring how CloudWatch alarms treat missing data(breaching / notBreaching)
- Amazon EC2 Auto Scaling events - Amazon EventBridge
- Run commands at launch (user data) - Amazon EC2
- AWS Systems Manager Run Command(`send-command`)
- Elastic IP addresses - Amazon EC2

- Using S3 Versioning
- Managing the lifecycle of objects - Amazon S3